Human Activity Recognition (CASAS) Executive Summary

Ambient sensor windows → activity predictions (XGBoost)

April 9, 2026

Table of contents

Executive Summary . . . 2
Problem & Goal . . . 2
Data & Labels . . . 2
Modeling Approach . . . 2
- Baseline Model . . . 2
- Primary Metric . . . 2
- Class Distribution . . . 3

Implementation (Selected Code) . . . 4
- Label Cleanup + Generalization (Polars) . . . 4
- Memory-Safe Sampling (Per Home) . . . 5
- Training Commands (XGBoost) . . . 5

Key Results . . . 5
Metrics (Saved Training Runs) . . . 6
- Best model (current leader) . . . 6

Risks, Limitations, and What Would Make It Better . . . 6
Future Improvements . . . 7

Dataset (post-cleaning)

9.19M windows

Sample fraction 1.0 = 9,186,574 labeled windows (30 homes).

Best model (macro-F1 CV)

0.777

Generalized target; sample fraction 0.40.

Targets

11 vs 41 classes

Generalized labels outperform specific labels on macro-F1.

Executive Summary

This project builds a multiclass classifier to recognize daily activities in smart homes from ambient sensor streams (motion, doors, light, etc.). Each training instance is a **feature vector computed over a sliding window of 30 sensor events**, and the model predicts either a fine-grained activity label (41-class) or a generalized activity category (11-class).

The clearest takeaway from evaluation results is that **generalized labels are substantially more learnable and more stable**, improving macro-F1, weighted-F1, and accuracy in cross-validation. The strongest saved run reaches **macro-F1 (CV) = 0.777** on the generalized target.

Problem & Goal

Problem: Given a feature vector summarizing the most recent sensor activity in a home, predict what activity is occurring.

Goal: Produce a model that is accurate enough for downstream use (monitoring, summarization, alerting, or analytics) and robust enough to run under local compute constraints.

Data & Labels

- Source: CASAS smart home ambient sensor dataset (homes `csh101`–`csh130`).
- Instances: sliding window of 30 sensor events, summarized into numeric features.
- Features: 36 numeric features per row (plus label columns).
- Total labeled windows used for modeling after label cleanup and filtering: **9,186,574**.
- Labels:
  - `activity`: specific activities (41 classes in current pipeline).
  - `activity_general`: 11 generalized categories (Hygiene, Leisure, Meal_Prep, etc.).

Generalization is performed by mapping multiple specific labels into a smaller set (e.g., `Cook_Breakfast`, `Cook_Lunch`, `Cook_Dinner` → `Meal_Prep`).

Modeling Approach

Baseline Model

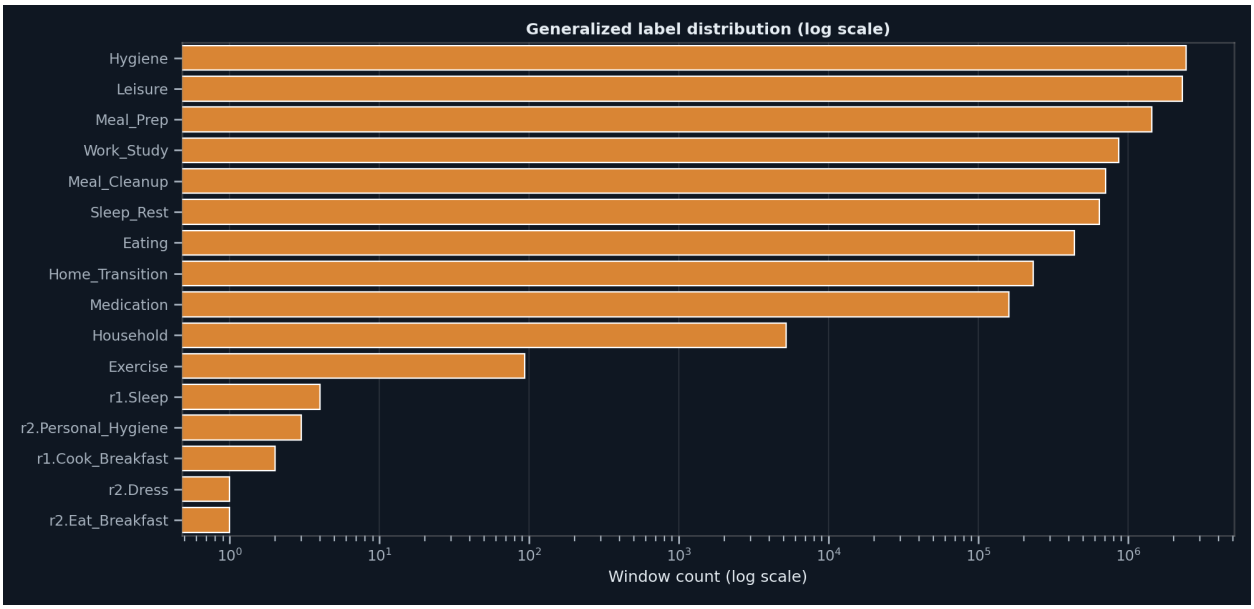
This work uses **XGBoost (gradient boosted decision trees)** because it: - handles heterogeneous numeric features well without normalization, - scales to large tabular datasets efficiently (`tree_method="hist"`), - supports multiclass probabilities (needed for log loss and calibration).

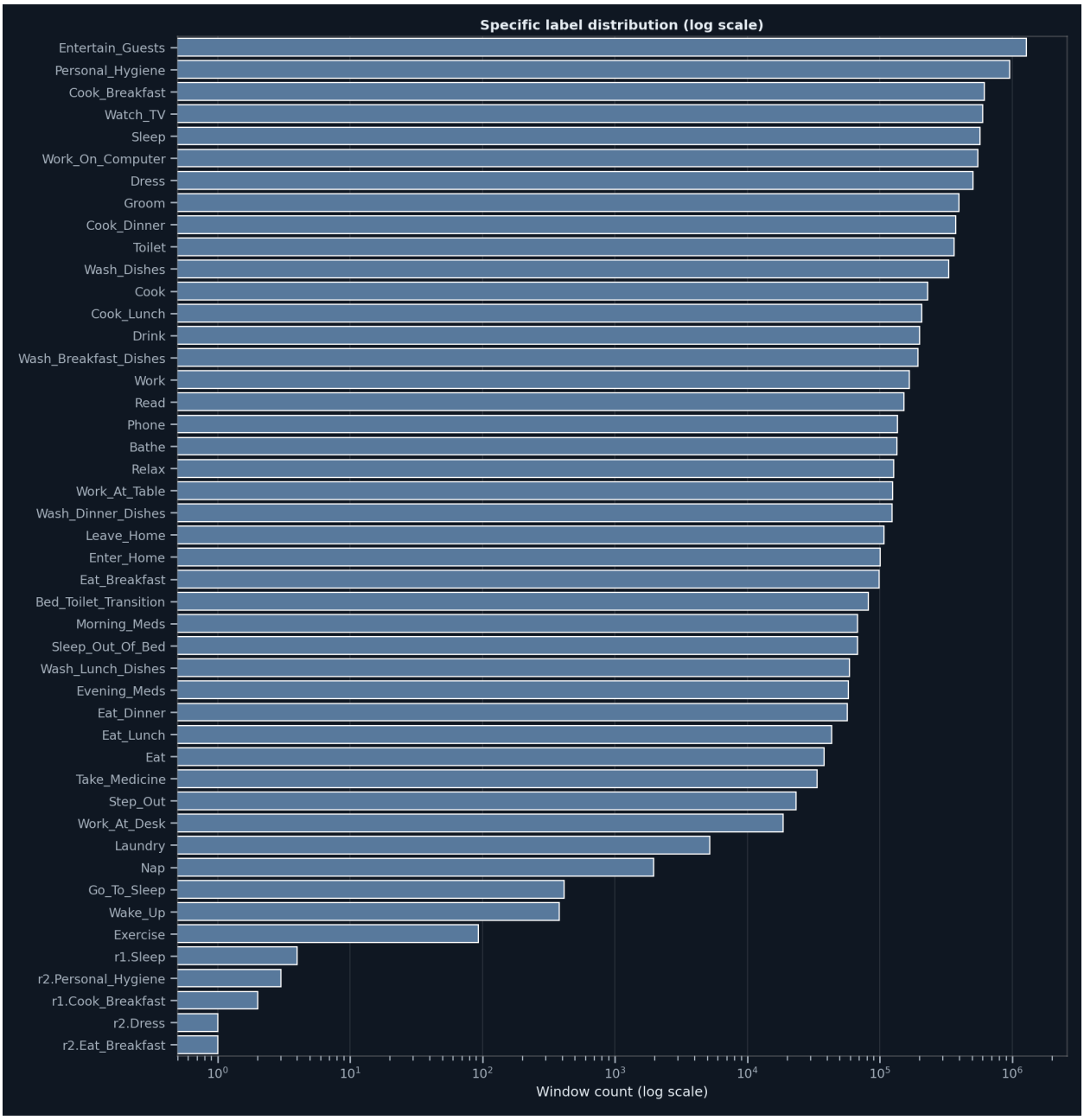
Primary Metric

The primary metric is **macro-F1** because the class distribution is imbalanced and each class should contribute equally. Accuracy and weighted-F1 are treated as secondary diagnostics.

Class Distribution

The following figures show label frequency (log scale) across all homes after label cleanup and filtering.





Implementation (Selected Code)

These are short excerpts from the training pipeline (`scripts/train_final.py`).

Label Cleanup + Generalization (Polars)

```
# Normalize odd labels like r1.Sleep / r2.Dress and add a generalized target column.
df_home = (
```

```
    pl.scan_csv(path)
    .with_columns(pl.col("activity").str.replace(r"^r[12]\.", "").alias("activity"))
    .filter(pl.col("activity") != "Other_Activity")
    .with_columns(pl.col("activity").replace(general_map).alias("activity_general"))
)
```

Memory-Safe Sampling (Per Home)

```
# Collect per-home then sample, to avoid materializing all homes at once.
df_home_sampled = df_home.collect().sample(fraction=0.30, seed=seed, shuffle=True)
```

Training Commands (XGBoost)

```
# Train on generalized labels (recommended default from baseline results)
python scripts/train_final.py --api native --target-column activity_general \
  --sample-fraction 0.05 --n-folds 5 --n-trials 12 --reserve-threads 4 --verbosity 2

# Alternate branch via sklearn wrapper
python scripts/train_final.py --api sklearn --target-column activity_general \
  --sample-fraction 0.05 --n-folds 5 --n-trials 12 --reserve-threads 4 --verbosity 2
```

Key Results

The table below compares a lightweight baseline evaluation for the two target choices:

target	n_classes	cv_f1_macro_mean	cv_f1_macro_std	cv_f1_weighted_mean	cv_accuracy_mean
Generalized activities	11	0.7198	0.0156	0.7838	0.7921
Specific activities	41	0.6566	0.0021	0.6990	0.7083

Interpretation: Generalized labels improve macro-F1, weighted-F1, and accuracy. This supports generalized activity prediction as the primary target when reliability is the priority.

Metrics (Saved Training Runs)

The table below is auto-generated from the saved run metadata in `models/meta_*.json` (refresh via `python docs/executive_summary/tools/generate_model_metrics_md.py`). It reflects the **primary CV score** used during training-time model selection (macro-F1). Sample fraction is a per-home sampling rate used to manage compute; **sample fraction 1.0 corresponds to 9,186,574 windows** after label cleanup and filtering.

Best model (current leader)

- Model: **XGBoost native**
- Target: **Generalized activities (11 classes)**
- CV macro-F1: **0.7772**
- Sample fraction: **0.4**
- Train macro-F1: 0.8088; train accuracy: 0.8304
- Boosting: 900 rounds (early_stopping_rounds=40)
- Key settings: learning_rate=0.030, max_depth=6, rounds=900

Showing top 5 runs (by CV macro-F1).

Model	Target	Sample fraction	Macro-F1 (CV)	Macro-F1 (train)	Accuracy (train)
XGBoost native	**Generalized activities (11 classes)**	0.4	**0.7772**	0.8088	0.8304
XGBoost sklearn	Generalized activities (11 classes)	0.01	0.6221	0.7607	0.8638
XGBoost sklearn	Generalized activities (11 classes)	0.01	0.6157	0.7607	0.8638
XGBoost native	Generalized activities (11 classes)	0.005	0.5938	0.7583	0.8563
XGBoost native	Generalized activities (11 classes)	0.005	0.5829	0.7615	0.8591

Risks, Limitations, and What Would Make It Better

- **Leakage risk:** adjacent windows are highly correlated; random row splits inflate metrics.
- **Cross-home generalization:** models can learn home-specific patterns; holding out homes is a stronger test.

- **Compute constraints (memory):** end-to-end training and cross-validation on the full dataset can exceed available RAM; experiments therefore rely on per-home sampling and limited hyperparameter searches.
- **Extreme class imbalance:** some specific labels are extremely rare; evaluation variance can be misleading.
- **Interpretability:** boosted trees are interpretable at the feature level, but not causal; confusion matrices matter.

Opportunities for improvement: - Switch evaluation to `GroupKFold` by `home_id` (or leave-one-home-out). - Report both macro-F1 and log loss, plus a confusion matrix for top activities. - Consider a two-stage model: generalized prediction first, then specific within a bucket.

Future Improvements

Future work focuses on strengthening generalization and interpretability: - Evaluate with an "unseen home" protocol (grouped by `home_id`) to avoid optimistic scores from correlated sliding windows. - Add confusion matrices and per-class support to make failure modes clear to non-technical stakeholders. - Consider a two-stage approach: predict generalized categories first, then predict fine-grained activities within each category.